

Aide

A voice-native platform for blind and visually impaired Nigerians to find work, prove their skills, and get paid — without a screen or keyboard.

PRD v2 Internal

Event: APIConf × Monnify Developer Challenge (#APIConfXMonnify) · **Team:** 2 · **Constraint:** Monnify Sandbox API only

Submission: ~12pm WAT, ~24–25 July 2026 (confirm exact time in the `apiconf-hackathon` Slack)

Stack: Next.js · TypeScript · Web Speech API · LLM agent (tool-calling) · Monnify · Tailwind

Read this first. This document is written so either teammate can build their half without asking questions. The money layer is already proven working against the live sandbox (see §12). The scope is deliberately tight: one voice loop, demoed end-to-end, nothing that doesn't appear on stage.

1. What we are building

Aide is a fully voice-controlled web app. A blind user talks to an agent — **Aide** — that finds jobs suited to them, applies on their behalf, runs a spoken skills assessment, and receives and manages their pay through the Monnify API. The user speaks; Aide listens (speech-to-text), reasons, acts, and speaks back (text-to-speech). Nothing in the user's journey requires sight or a keyboard.

2. The problem

A blind Nigerian who can transcribe, translate, code, or do phone support is locked out of digital income — not by ability, but by interface. Job boards, application forms, skills tests, and the payment dashboards where money lands all assume you can see a screen and use a keyboard. The capability exists; the access does not.

The one-sentence pitch: "A blind graduate who transcribes faster than you can't finish a single job application — because every form, test, and payment screen assumes she can see it."

The enemy: the screen-and-keyboard assumption baked into every fintech and job platform.

3. Users

User	Who they are	Interface
Primary	Blind / severely visually impaired Nigerian adults seeking remote, task-based income (transcription, translation, audio QA, phone support, data labeling, simple coding).	100% voice
Low-vision	Users with partial sight who benefit from a bold, high-contrast, large-type screen alongside audio.	Voice + accessible visual
Secondary	Employers posting accessible, task-based jobs; need to verify a candidate's skill and pay them.	Standard web UI

Never assume the primary user can: see a screen, read an OTP, visually confirm an account number, or use a keyboard reliably.

4. Product decisions (locked)

4.1 "Voice-native" means the voice path is complete, not that no screen exists

The blind user can do *everything* by voice — sight is never required. A visual layer still exists, but it is not the primary user's dependency. It serves (a) low-vision users, with bold high-contrast type, and (b) the demo: an **observer screen** that shows the live transcript, current state, balance, and payment events so judges can *see* the loop working, not just hear it.

4.2 Delivery: web app + microphone (not telephony)

We build a browser app with a mic button. Telephony (call a number, no app) is the stronger long-term product and a great roadmap line, but it is the wrong build for an 8-day window: it is mostly plumbing, adds on-stage failure points, and — critically — has no screen for judges to follow. Web + mic is fastest to build, rock-solid to demo, and the screen doubles as the observer view. **We pitch telephony as "what's next."**

5. Scope

Must-have — this is the demo, build first

1. **Voice I/O loop** — speech-to-text in, Aide reasons, text-to-speech out. < ~2s per turn.
2. **Aide agent** with a fixed toolset (§7). Conversational on navigation, deterministic on money.
3. **Job discovery + application** — a seeded set of accessible jobs; Aide reads them, the user picks, Aide applies.
4. **Oral assessment** — Aide administers a short spoken skills test on the employer's behalf, scores it, marks the skill verified.

5. **Payment receipt (PROVEN)** — employer funds land in the user's Monnify reserved account; Aide announces it only after server-side verification.
6. **Withdrawal** — user says "send my money to my bank"; Aide does a name-enquiry read-back, then calls Monnify's disbursement (see §9 for demo vs production behaviour).

Nice-to-have — only if the loop above is solid

- Application tracking by voice · balance/history queries · employer web portal to post jobs and confirm completion · platform fee via transaction splitting.

Cut without hesitation

- Real accounts/KYC at scale · native mobile app · recurring/bulk transfers · multi-language (English + optional Pidgin only) · anything needing live Monnify keys.

6. Core user flows

F1 — Find work. "Aide, find me transcription jobs." → Aide lists 2–3 seeded jobs with pay + task. → "Apply to the second one." → Aide confirms and applies.

F2 — Prove the skill. "This job needs a transcription test. I'll play 20 seconds of audio; repeat what you hear." → captures the response → scores it → marks the skill **verified**.

F3 — Get paid. Employer marks the task complete and funds the reserved account (in the demo, via the Monnify web simulator). → Monnify confirms the payment → Aide: "You've been paid ₦12,000 for the transcription job."

F4 — Withdraw. "Send it to my bank." → Aide validates the account (name enquiry) and reads the name back for spoken confirmation → on "yes," calls the disbursement API → confirms status.

7. The Aide agent

Aide is an LLM with strict tool-calling. It never invents facts about money, jobs, or balances — it calls a typed server tool and speaks the result. The tools are the only actions it can take.

Tool	Does	Backed by
<code>listJobs(filter)</code>	Return seeded jobs matching a spoken filter	DB
<code>applyToJob(jobId)</code>	Create an application for the user	DB
<code>startAssessment(jobId)</code>	Begin the oral skills test	DB + audio
<code>submitAssessment(answer)</code>	Score the spoken answer, set skill verified	DB + scoring
<code>getBalance()</code>	Return verified balance (sum of PAID inbound)	Monnify
<code>getApplicationStatus()</code>	List the user's applications and states	DB
<code>registerPayoutAccount(acct, bank)</code>	Name-enquiry + save the user's bank for read-back	Monnify
<code>initiateWithdrawal(amount)</code>	Call Monnify single-transfer	Monnify
<code>confirmPayment()</code>	Poll/verify a landed payment before announcing	Monnify

Guardrails (these are also scoring points — the challenge says "AI slop is frowned upon"):

- Aide reports a payment, balance, or transfer **only** from a verified server call — never from its own reasoning or a raw webhook. The LLM narrates; it never decides financial truth.
- Every money-moving action needs an explicit spoken confirmation, and Aide reads back the amount + destination name before executing.
- If speech-to-text is unclear on a money command, Aide re-confirms rather than guessing.

8. Payment architecture (Monnify)

The platform is the Monnify merchant. Each worker gets a dedicated virtual account (a reserved NUBAN) that acts as their earnings account. Employers pay into it; Monnify notifies us; we verify and let Aide announce.

Flow	Monnify primitive	Status
Per-user earnings account	Reserved Account (virtual NUBAN), created with contract code 4523534626	Proven
Employer pays worker	Inbound → webhook (deployed) or transaction polling	Proven
Verify before announcing	Signed webhook (SHA-512 HMAC) → server-side transaction re-fetch	Proven
Confirm the user's bank	Name enquiry / validate bank account	To verify on a real account
Withdrawal	Single Transfer (disbursement)	Gated — see §9
Balance	Reserved-account transactions	Proven

Verified endpoints (already wrapped in `src/monnify.ts`)

```

POST /api/v1/auth/login           → bearer token (~1h, cached)
POST /api/v2/bank-transfer/reserved-accounts → create earnings account
GET  /api/v1/bank-transfer/reserved-accounts/transactions → confirm inbound (PAID)
GET  /api/v2/transactions/{ref}    → re-verify a transaction
GET  /api/v1/disbursements/account/validate → name enquiry (read-back)
POST /api/v2/disbursements/single   → withdrawal (needs activation)
POST /api/v2/disbursements/single/validate-otp → authorize payout (server-side)

```

Sandbox mechanics: inbound "employer payments" are simulated via

`https://websim.sdk.monnify.com/?#/bankingapp` — send a transfer to the reserved account (bank must match), which produces a real transaction we confirm through the API. Base URL `https://sandbox.monnify.com`.

9. Withdrawal design — and why the user never touches an OTP

Outbound payouts are 2FA-protected and, on our sandbox, gated behind full business activation (KYC + Monnify review). We handle this with an architecture that is both correct for production and demoable now: **the platform (the merchant) authorizes the payout; the blind user only consents by voice.** A user who cannot read an OTP off a screen never has to.

	Demo now (no KYC)	After business activation
Transfer call	Real API call → returns <code>PENDING_AUTHORIZATION</code>	Real API call → platform authorizes OTP server-side
Aide says	"Your withdrawal is on its way — I'll confirm when it lands."	"Done. ₦12,000 sent to your GTBank account."
User handles an OTP?	No	No — the platform does

The security a blind user gets instead of a visual OTP: **(1)** voice consent, **(2)** Aide reads back the destination account name from name-enquiry so they *hear* exactly where the money is going, **(3)** the platform authorizes the settlement. "Security without sight."

10. Technical architecture

Layer	Choice	Why
Framework	Next.js (App Router) + TypeScript	One project for UI + API routes; deploys to Vercel; gives a public URL for webhooks.
Styling	Tailwind CSS	Fast, and easy to make the observer screen bold + high-contrast.
Speech-to-text	Web Speech API (<code>SpeechRecognition</code>); fallback Deepgram if browser quality is poor	Zero-setup, in-browser, fastest path to a working loop.
Text-to-speech	Web Speech API (<code>SpeechSynthesis</code>); optional ElevenLabs for a warmer voice	In-browser, no infra; upgrade voice only if time allows.
Agent	LLM with tool-calling — latest Claude (e.g. <code>claude-opus-4-8</code> / <code>claude-sonnet-5</code>)	Reliable structured tool-calling; tools are typed server functions.
Backend	Next.js API routes (Node runtime)	Hosts the agent tools + Monnify webhook receiver.
Database	Convex (realtime) <i>or</i> Postgres/Prisma (Neon)	Convex pushes live updates to the observer screen; Postgres if the team knows it cold.
Payments	Monnify sandbox, server-side only (<code>src/monnify.ts</code>)	Keys never touch the client.
Hosting	Vercel	Public URL for webhooks; preview deploys.
Auth	Magic link / pre-seeded demo users	Out of the critical path; do not over-build.

Data flow. Mic → STT → /api/agent (LLM + tools) → tool hits DB and/or Monnify server-side → result returned → TTS speaks it → observer screen updates live. Monnify webhook → /api/webhook/monnify → verify signature → re-fetch transaction → mark PAID → Aide announces.

11. Data model (minimum)

User	id, name, email, reservedAccountRef, reservedAccountNumber, payoutBank?, payout/
Job	id, title, task, skill, payAmount, employerName
Application	id, userId, jobId, status (applied assessed hired paid)
Assessment	id, applicationId, prompt, answer, score, verified (bool)
LedgerEntry	id, userId, type (credit debit), amount, monnifyRef, status, createdAt

Balance = sum of credit ledger entries with status PAID. Every entry is written only after Monnify confirms — the ledger is the single source of financial truth Aide reads from.

12. Proven — live-proof results (16 July 2026, real sandbox)

Step	Result
Auth	bearer token issued + cached
Create reserved account	real NUBAN 1000824409 (Wema), name "MONNIFY / byte tech-Aid"
Inbound payment via websim	₦12,000 → "Transaction Successful"
Confirm via API	PAID , ref MNFY 04 20260716104241 000067

The core mechanic — worker account → employer pays → Monnify confirms → Aide announces the verified amount → balance — works end-to-end today, with nothing mocked. Full detail in `PROOF.md`.

13. Accessibility principles (non-negotiable)

- No task requires sight or a keyboard to complete, end to end. Test the demo screen-off.
- Aide confirms before every irreversible/money action and reads back critical values.
- Every failure has a spoken, human explanation — never a silent dead-end.
- The visual layer is bold, high-contrast, large-type, and screen-reader friendly (for low-vision users).

14. Repository (what already exists)

```
monify/
  PRD.html / PRD.md      this document
  PROOF.md               live-proof results + gotchas
  .env                   Monnify sandbox keys (gitignored – never commit)
  src/
    env.ts               validated config
    monnify.ts            the client: auth, reserved accounts, verify, name enquiry, transfer, we
    proof.ts              end-to-end money-loop proof runner
    create-account.ts     mints a worker earnings account
    inbound.ts            confirms a real inbound payment via the API
    webhook.ts            signed webhook receiver
    npm run account | inbound | proof | webhook
```

The functions in `src/monnify.ts` become Aide's money tools directly — no rewrite needed.

15. Risks & mitigations

Risk	Severity	Mitigation
Payout blocked by KYC activation	High	Scope demo to inbound; show withdrawal as pending-authorization (real call). Pursue activation in parallel only if safe.
Browser STT mishears money commands	High	Mandatory read-back + spoken confirmation before executing.
Aide "hallucinates" a payment	High	Money facts only from verified server calls; never from the model or a raw webhook.
Voice latency kills the demo	Medium	Pre-warm the model, stream TTS, keep turns short, rehearse.
Over-scoping	Medium	Ship F1–F4 first; everything else is nice-to-have.

16. Timeline (from 16 July; deadline ~24–25 July)

Day	Focus
1 (done)	De-risk money loop — reserved account, real inbound payment, API confirmation. Complete.
2–3	Money spine as Aide tools: reserved-account-per-user, verified payment announcement, ledger, withdrawal (pending) with name-enquiry read-back.
3–4	Voice loop + agent: STT/TTS, Aide with the \$7 toolset, job list + apply.
5	Oral assessment + skill verification.
6	Observer screen, polish, error handling, screen-off test.
7	Rehearse demo, record backup video, README, social post (#APIConfXMonnify #DeveloperChallenge), submit early.

Build days ÷ 3 = real days after life happens — the buffer is intentional.

17. Team split (2 people)

Person A — Money & backend	Person B — Voice, agent & demo
Monnify integration (already started), webhook receiver, transaction verification, ledger, withdrawal flow, deploy + public URL, DB schema.	STT/TTS voice loop, Aide agent + tool wiring, job & assessment flows, observer screen, pitch + rehearsal (owns the mic).

18. Submission checklist (missing a deliverable = auto-loss)

- ☐ Team of 2 registered in the `apiconf-hackathon` Slack
- ☐ Built on the Monnify **sandbox** API only
- ☐ Public, documented repository
- ☐ Live demo + recorded backup video (with screenshots)
- ☐ Social post with `#APIConfXMonnify` and `#DeveloperChallenge`
- ☐ Submitted via the Slack channel link before 12pm WAT on the deadline